Session 6 : Load Balancing & CDN

RECAP: Session 5 : Cloud Networking & VPC

1. Cloud Networking Overview: The network as the deployment boundary; core building blocks
2. Virtual Private Cloud (VPC): Logically isolated networks; VPC concepts across providers
3. Subnets & IP Addressing: CIDR notation & subnet sizing; public vs private subnets; availability-zone placement
4. Routing & Gateways: Route tables; internet gateway; NAT gateway; VPC peering & private connectivity
5. Network Security Controls: Security groups (stateful) vs network ACLs (stateless); least-privilege network design

Agenda for Today's Session

1. Load Balancing Basics: Purpose: availability & scalability; the single-point-of-failure problem
2. Load Balancer Types: Layer 4 (transport) vs Layer 7 (application); internal vs internet-facing
3. Distribution Algorithms: Round robin; least connections; IP hash; weighted distribution
4. Health Checks & Sessions: Health check design; draining an unhealthy backend; sticky sessions vs stateless design
5. DNS & Content Delivery: DNS-based routing & failover; CDN: edge caching, origin & cache invalidation

---------- SECTION 1: 1. Load Balancing Basics: Purpose: availability & scalability; the single-point-of-failure problem ----------

Slide 1: The Single Point of Failure Problem

Heading: Single Point of Failure

Subheading: The Core Problem

Content:

A single server gives a web application only one path for requests, so its failure stops everything. Cloud architectures fix this by placing multiple servers behind a traffic distributor.

- Single point of failure: one component whose failure stops the entire application.
- Hardware failure, an operating-system patch, or a crash removes the only server.
- Traffic above the server's capacity slows or drops requests, since no second server exists.
- A bigger server raises capacity but never removes the single point of failure.

Analogy: A railway station with only one ticket counter forces every passenger to wait once that counter closes for a break — exactly as a single server failing removes the only route available for every user's request.

Visual Guidance: Left: one browser icon with an arrow into one server box, a red cross through it. Right: the same browser with arrows into two server boxes, one still green.

Speaker Notes: "Start by stating that one server means one failure point, linking it to the earlier horizontal-scaling idea. Use the ticket-counter analogy to make the outage concrete before introducing load balancing. Interviewers often ask what a single point of failure means — this slide answers that."

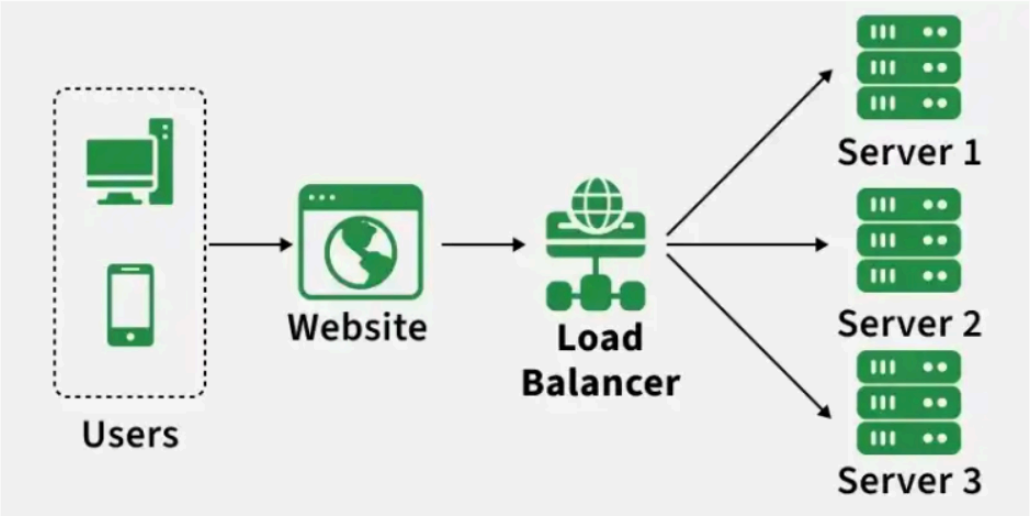
Slide 2: What a Load Balancer Does

Heading: Traffic Distribution

Subheading: Load Balancer Role

Content:

A load balancer is a component that receives all incoming requests and distributes them across multiple backend servers, so no single server is overloaded or becomes a single point of failure.



- Identical servers behind the load balancer let one fail without downtime.

- Adding or removing backend servers changes capacity without changing application code.
- AWS offers this as Elastic Load Balancing, set up from the EC2 console.
- An Application Load Balancer is billed hourly plus traffic processed.
- Choosing a load balancer type is also a cost decision.

Visual Guidance: One load-balancer icon in the middle, arrows fanning out to three server icons below it; label the top arrow 'incoming requests'.

Speaker Notes: "Explain the load balancer as the layer between users and servers, deciding where each request goes. Note that this is the exact service used later in the practical, reachable from the EC2 console. A common interview question asks why users cannot get a fixed server address directly — this mechanism is the answer."

Slide 3: Purpose: Availability and Scalability

Heading: Availability and Scalability Goals

Subheading: Why It Exists

Content:

Removing the single point of failure is not the only benefit; the same routing that avoids failed servers also lets capacity grow by adding more servers.

- Availability: traffic keeps reaching healthy servers during instance failure or patching.
- Scalability: new servers join the load balancer as demand grows, with no downtime.
- Horizontal scaling works safely only when users reach the load balancer, not a fixed server address.
- Extra backend servers for availability add compute cost, a trade-off covered later.

Visual Guidance: Simple timeline: two servers growing to four behind the same load balancer icon, with a small cost-tag icon beside the fourth server.

Speaker Notes: "Stress that availability and scalability both come from rerouting traffic, applied to failure versus growth. Mention that more servers mean more cost, without a pricing discussion. Note that distributing traffic needs a defined method, which the next section covers."

---------- SECTION 2: 2. Load Balancer Types: Layer 4 (transport) vs Layer 7 (application); internal vs internet-facing ----------

Slide 4: Load Balancer Types Overview

Heading: Load Balancer Types

Subheading: Two Classification Axes

Content:

Load balancers are classified in two ways: by the OSI layer they work at, and by who can reach them. Both choices change routing intelligence, speed, and exposure.

- Layer axis: Layer 4 balancers route by IP and port; Layer 7 balancers read HTTP content
- Internet-facing: has a public IP and accepts requests from users on the internet.
- Internal: has only a private IP and routes traffic between services inside the private network.
- A real deployment picks one from each axis, like an internet-facing Layer 7 balancer.
- By deployment: load balancers can also be hardware appliances, software (like NGINX or HAProxy), or cloud-managed services (like AWS ELB); this session focuses on cloud-managed.

Visual Guidance: 2x2 grid: rows Layer 4 / Layer 7, columns Internet-facing / Internal, each cell naming a typical use case.

Speaker Notes: "Set up both axes before going deep into either one. Say plainly: layer decides how smart the routing is, exposure decides who can send traffic. Mention this map will be filled in across the next three slides."

Slide 5: Layer 4 vs Layer 7 Balancing

Heading: Transport Vs Application

Subheading: Balancer Comparison

Content:

Layer 4 balancers forward packets using only transport headers, while Layer 7 balancers terminate the connection and read the full HTTP request. AWS calls these Network Load Balancer (NLB) and Application Load Balancer (ALB).

Aspect	Layer 4 (transport)	Layer 7 (application)

Decision basis	Source/destination IP and port number	URL path, HTTP headers, cookies
Visibility into request	Cannot read request content	Reads full HTTP request and response
Speed and overhead	Lower overhead, no content parsing	Higher overhead, content inspection
Routing capability	Single destination pool per listener	Rule-based routing to different pools by path or host
Typical AWS service	Network Load Balancer (NLB)	Application Load Balancer (ALB)

Visual Guidance: Side-by-side packet diagrams: Layer 4 arrow passing through unopened; Layer 7 arrow opened to show HTTP headers before forwarding.

Speaker Notes: "Layer 4 is faster since it never opens the packet; Layer 7 is smarter since it inspects HTTP data. Note the AWS names, as they appear on the console. Interview hook: ask when NLB beats ALB for raw TCP/UDP speed and preserving client IP."

Slide 6: Internal vs Internet-Facing

Heading: Internal Vs Internet-Facing

Subheading: Who Can Reach It

Content:

Load balancers are also classified by exposure: internet-facing balancers get a public IP and accept internet traffic, while internal balancers get only a private IP, reachable within their VPC.

- Internal balancers connect internal tiers, like an application calling a backend service.
- Internal balancers shrink the attack surface since no public route exists.
- Exposure type and layer type are independent; both can be Layer 4 or Layer 7.

Analogy: An office building has a public reception desk anyone off the street can walk into, and separate staff-only corridors reachable only by employees already inside — just as an internet-facing balancer accepts any public visitor while an internal balancer accepts only traffic that already originates inside the VPC.

Visual Guidance: Diagram: public internet arrow hitting an internet-facing balancer icon with a public IP; a second balancer icon inside the VPC boundary with only internal arrows reaching it.

Speaker Notes: "Stress that exposure is separate from layer, not a variant of it. Use public reception versus staff-only corridor as the everyday phrasing. Exam hook: ask why making a database-facing balancer internal-only is good security practice."

Slide 7: Choosing Load Balancer Type

Heading: Choosing Balancer Type

Subheading: Combining Both Axes

Content:

A typical architecture places an internet-facing Layer 7 balancer in front of user traffic and an internal Layer 4 or 7 balancer between services, matching intelligence to each hop.

- AWS provides an ALB for Layer 7 traffic and an NLB for Layer 4.
- The internet-facing or internal scheme is set once, in the EC2 console.
- Public websites typically use an internet-facing ALB; internal calls use an NLB or ALB.
- An ALB receives traffic through a listener (a protocol and port) and forwards it to a target group, a named set of backend servers that share a health check

Visual Guidance: Architecture diagram: public internet to internet-facing ALB, ALB fans to a service tier, service tier to an internal NLB, internal NLB fans to a database tier.

Speaker Notes: "Tie both axes together with one concrete architecture to make the choice practical. Point to the EC2 console's load balancer creation screen as where this setting lives. Note that an algorithm still decides which server gets each request once traffic reaches the right balancer."

---------- SECTION 3: 3. Distribution Algorithms: Round robin; least connections; IP hash; weighted distribution ----------

Slide 8: Distribution Algorithms Landscape

Heading: Distribution Algorithms

Subheading: Four Approaches Compared

Content:

A load balancer needs a rule, called an algorithm, to pick a healthy target for each request. Algorithms fall into two types: static algorithms follow fixed rules and ignore server load, while dynamic algorithms check live server state before deciding. Many variations exist, but four are used most widely.

- Round robin (static): cycles through targets in fixed order, assuming equal capacity.
- Least connections (dynamic): picks the target with the fewest active connections.

- IP hash (static): maps a client's IP address to the same target, as long as the server pool stays the same.
- Weighted distribution (static or dynamic): adds capacity weights to round robin or least connections, so stronger targets get more traffic.

Visual Guidance: Four-quadrant diagram: load balancer in the centre, one arrow style per algorithm (cyclic, counter-based, hash-based, weighted) fanning out to target icons.

Speaker Notes: "Frame this as the rulebook a load balancer consults for every request. Stress that none of the four is universally best; each suits a different traffic shape. Interviewers may ask which algorithm fits a scenario, so link each name to its trigger."

Slide 9: Round Robin And Weighting Mechanism

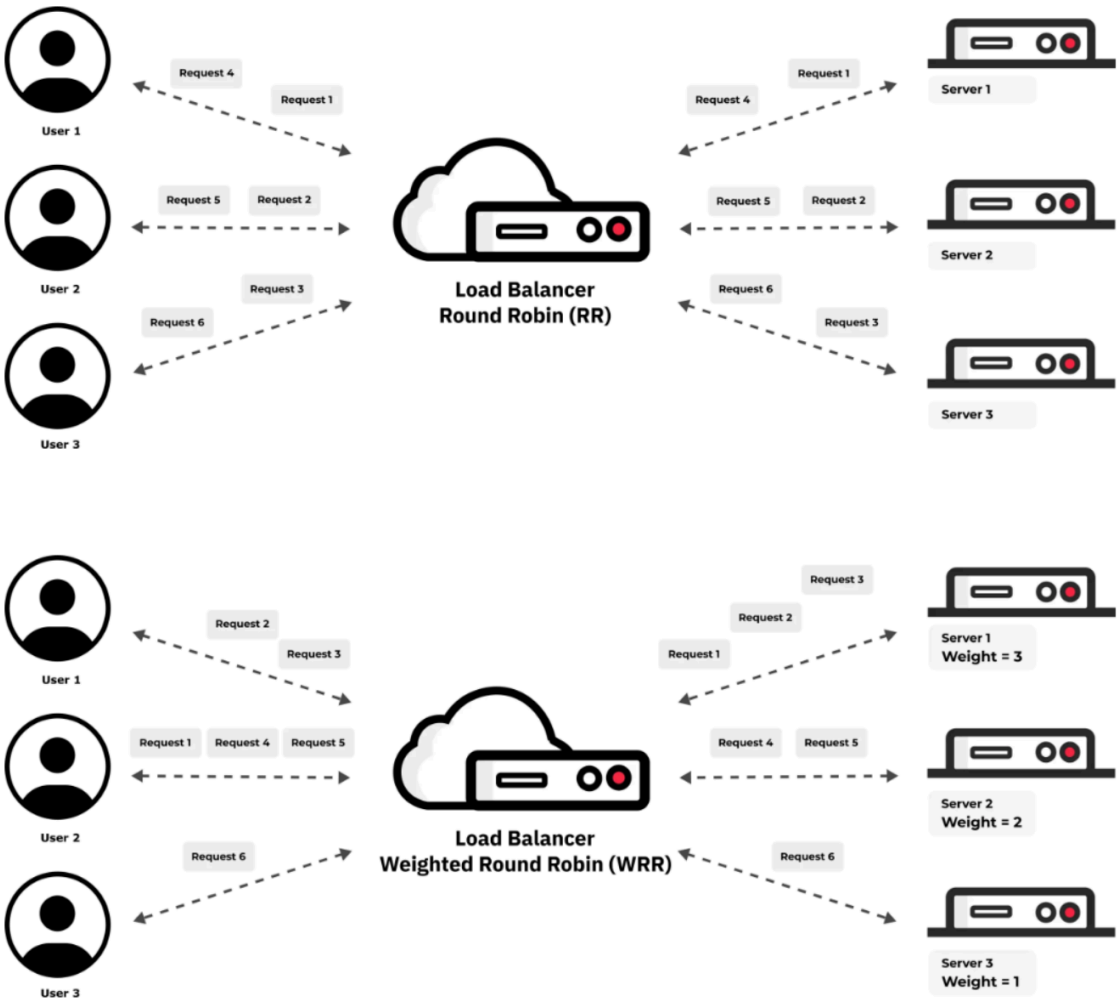
Heading: Round Robin Weighting

Subheading: Static Capacity Bias

Content:

Plain round robin treats all targets equally, which fails when server capacity differs. Weighted distribution gives stronger targets a proportionally larger share of the rotation.

- Smooth weighted round robin increases each target's running total by its weight, avoiding bursts.
- A 3:1 weight ratio averages three requests to one target per other request.
- On AWS ALB, the default routing algorithm within a target group is round robin. Weighted target groups can be configured in listener rules to distribute traffic between target groups (for example, 90:10). These are two different mechanisms.
- Weights are set once from known capacity, not changed automatically with load.



Visual Guidance: Sequence strip showing a weighted rotation order (e.g. A A A B) under two server icons sized differently to show capacity.

Speaker Notes: "Explain smoothing via the running-total idea so the order isn't clumped — a common exam trip-up. Note that weights are manual and planned, unlike least connections which reacts automatically. Mention AWS target group weight field as the real-world example."

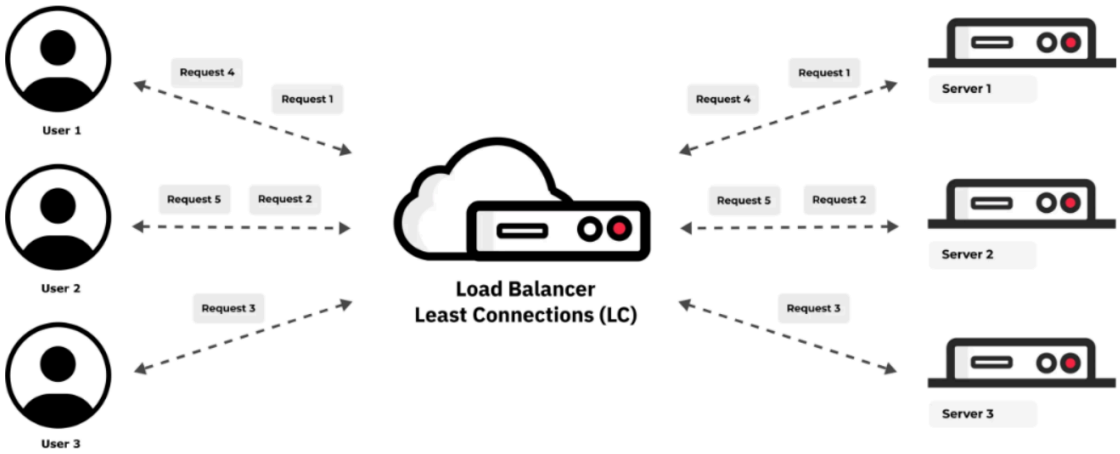
Slide 10: Least Connections In Depth

Heading: Least Connections Depth

Subheading: Dynamic Load Tracking

Content:

Round robin and weighting follow a fixed plan, but least connections reacts continuously, sending each new request to whichever target is least busy.



- A per-target counter rises when a request starts and falls on completion.
- Long-lived connections let least connections outperform round robin under uneven request durations.
- AWS ALB's least outstanding requests option, selected in the target group settings, counts in-flight requests, not raw connections.
- No manual tuning is needed; counts adjust as servers speed up or slow down.

Visual Guidance: Two target icons with live counters (e.g. 2 and 5) and an arrow from the load balancer pointing to the lower counter.

Speaker Notes: "Emphasise that the counter is live, updating on every request start and finish, unlike a fixed weight. Use an upload example to show why duration matters more than request count. Remember the AWS term, least outstanding requests, for interviews."

Slide 11: IP Hash For Client Affinity

Heading: IP Hash Explained

Subheading: Client To Server Mapping

Content:

Some applications keep client state only on the server that first handled a request, so later requests must return there. IP hash hashes the client's source IP to pick a fixed target.

- Adding or removing a target reshuffles mapping, redirecting many clients at once.

- Clients sharing one corporate or mobile-network gateway IP can overload a single target disproportionately.
- AWS has no setting named IP hash: ALB uses cookie-based stickiness; NLB offers source-IP stickiness
- IP hash avoids shared session stores, unlike server-side replication methods.

Analogy: In an exam, each student's hall is decided by their roll number (for example, the last digit decides the hall), so the same student always goes to the same hall. In the same way, IP hash uses the client's IP address to always route that client to the same backend server.

Visual Guidance: Three client icons with distinct IPs, each with a dashed line always landing on the same one of two target icons, labelled 'hash(IP)'.

Speaker Notes: "Introduce the session-affinity problem before naming IP hash. Flag the reshuffle risk when targets change, since interviewers often ask about it. Mention Network Load Balancer's flow hashing as the transport-layer equivalent."

---------- SECTION 4: 4. Health Checks & Sessions: Health check design; draining an unhealthy backend; sticky sessions vs stateless design ----------

Slide 13: Health Checks And Session Handling Map

Heading: Health And Sessions

Subheading: Section Topic Map

Content:

A load balancer must know which backends are actually working, and decide how a user's follow-up requests are handled across multiple servers. Three linked questions cover this.

- Health checks: is this backend working? The load balancer tests each backend regularly.
- Draining: how is a backend removed safely? Existing requests finish before it is taken out.
- Sessions: where do a user's next requests go? Sticky sessions send them to the same backend; stateless design lets any healthy backend serve them.

Visual Guidance: Three labelled boxes in a row: Health Checks, Draining, Sessions — each with a one-icon hint (heartbeat, tap closing, cookie).

Speaker Notes: "Present these three as one connected story: detect failure, remove backend safely, then handle returning users. Note this builds on the distribution algorithms already covered. Exam angle: expect a question distinguishing health checks from session stickiness."

Slide 14: Designing A Health Check

Heading: Health Check Design

Subheading: How Checks Work

Content:

A health check works like a repeated vitals check: the load balancer sends a test request and reads the reply, confirming the application is actually responding, not just running.

- Interval: sets how often the check repeats; the ALB default is every 30 seconds
- Timeout: longest time the backend is given to reply before a miss.
- Healthy threshold: consecutive passes needed before traffic resumes to a backend.
- Unhealthy threshold: consecutive failures needed before traffic stops to a backend.

Analogy: A doctor does not assume a patient is fine just because they are breathing; a short, repeated vitals check confirms the body is actually functioning — just as a health check confirms the server is actually serving requests correctly, not merely running.

Visual Guidance: Timeline strip showing repeated check pulses at fixed intervals, with a counter of consecutive passes/fails building toward the two thresholds.

Speaker Notes: Stress that a server can be powered on yet still fail a health check if the application crashes while the OS keeps running. Use 'repeated vitals check' before naming the thresholds. Interview hook: ask what happens if the interval is too long or too short, since that trades detection speed against false alarms.

Slide 15: Draining An Unhealthy Backend

Heading: Connection Draining

Subheading: Removing A Backend Safely

Content:

Instant removal would cut off in-flight requests. Draining gives the backend a grace period, the deregistration delay, to finish current work before leaving rotation.

- An unhealthy backend stops getting new traffic at once; when it is deregistered (for example, replaced by Auto Scaling), draining lets its in-flight requests finish.

- New requests stop reaching the backend as soon as deregistration starts (manual removal, scale-in or a deployment).
- In-progress requests on that backend complete normally during the grace period.
- The deregistration delay is a target group setting; the AWS default is 300 seconds.
- Once the delay expires, any remaining connections are closed.

Visual Guidance: Backend box shown in three states left to right: serving, draining (fading, with two in-flight arrows still reaching it), removed.

Speaker Notes: "Draining is the polite way to retire a server, unlike yanking it out mid-request. It also runs during planned scale-down, not just after a failed health check. Exam angle: draining exists to avoid breaking in-flight requests."

Slide 16: Sticky Sessions Versus Stateless Design

Heading: Sticky Vs Stateless

Subheading: Two Session Approaches

Content:

HTTP is stateless by default, so a load balancer must decide whether a returning user must land on the same backend, or whether any backend can serve them equally well.

Approach	How It Works	Trade-off
Sticky sessions	A cookie set by the load balancer binds a user to one backend for the session duration	Breaks if that backend drains or fails, and skews load unevenly
Stateless design	Session data lives outside the server, so any healthy backend can answer any request	Needs an external store, but removal and scaling stay simple

Visual Guidance: Side-by-side diagram: left shows one user cookie-pinned to one server box; right shows the same user's requests landing on either server with a shared data store below both.

Speaker Notes: "Link this to draining: a sticky user on a draining backend forces a remap. Stateless design is preferred since it lets instances be added or removed freely. Interview hook: stateless design scales better because it removes server-affinity bottlenecks."

---------- SECTION 5: 5. DNS & Content Delivery: DNS-based routing & failover; CDN: edge caching, origin & cache invalidation ----------

Slide 17: DNS and CDN for Internet-Scale Delivery

Heading: DNS And CDN

Subheading: Routing And Delivery

Content:

Load balancing spreads traffic within one location. DNS-based routing and content delivery solve the same problem at internet scale, across regions and distant users.

- DNS-based routing returns different endpoint addresses based on a policy like location, latency or health.
- DNS failover swaps a failed address for a healthy one, but only after cached DNS answers expire (TTL).
- A Content Delivery Network (CDN) caches copies of content at edge locations close to users.
- Edges fetch content from the origin on a cache miss; cache expiry (TTL) and invalidation keep copies up to date.

Visual Guidance: Map diagram: a user icon, a DNS resolver choosing between two regional data centres, and separately a user hitting a nearby CDN edge server linked back to one origin server.

Speaker Notes: "Frame this as traffic steering at a wider scale than load balancing. DNS decides which region a user reaches; the CDN decides where content is stored. Interviewers often ask how DNS and CDN differ from a load balancer."

Slide 18: Steering Traffic Through DNS

Heading: DNS Based Routing

Subheading: Steering By Resolution

Content:

Domain Name System (DNS) normally returns one fixed address for a name, but DNS-based routing returns different addresses based on policy. This steers users to a region without the user noticing any choice.

- Geolocation routing: returns the record assigned to the user’s country or continent; geoproximity picks the nearest location.
- Latency-based routing: picks the region that historically responded fastest to the user.

- Weighted routing splits DNS answers across regions by a configured traffic percentage.
- Multi-value answer routing: returns several healthy addresses for the client to choose.

Analogy: When you order from a restaurant chain on a food delivery app, the app automatically sends your order to the branch nearest to you or the one that is open and least busy, even though you only searched the restaurant's name. In the same way, DNS-based routing takes one domain name and returns the address of the regional server best suited for each user.

Visual Guidance: World map with a DNS resolver icon branching arrows to three region icons, each arrow labelled geo, latency or weighted.

Speaker Notes: "Explain that the same name can resolve to different addresses depending on who asks and from where. Use the enquiry-counter analogy before naming the four policy types. Interviewers may ask how this differs from a load balancer: routing happens before any connection opens."

Slide 19: Detecting and Routing Around Failures

Heading: DNS Failover Mechanism

Subheading: Health Checked Record Swap

Content:

Proximity routing alone cannot sense a region is down, so DNS failover adds continuous health checks that stop including a failed address in new DNS answers.

replace

- Route 53 failover routing sends traffic to a primary record, switching to a secondary if it fails.
- A record failing consecutive checks is marked down and removed from DNS answers.
- Failover completes after detection plus TTL expiry of cached answers.
- Lower TTL values speed up failover but raise resolver query load.

Visual Guidance: Timeline strip: health checker pinging a region every interval, a red cross on the region after repeated failures, then the DNS answer list updating to drop it.

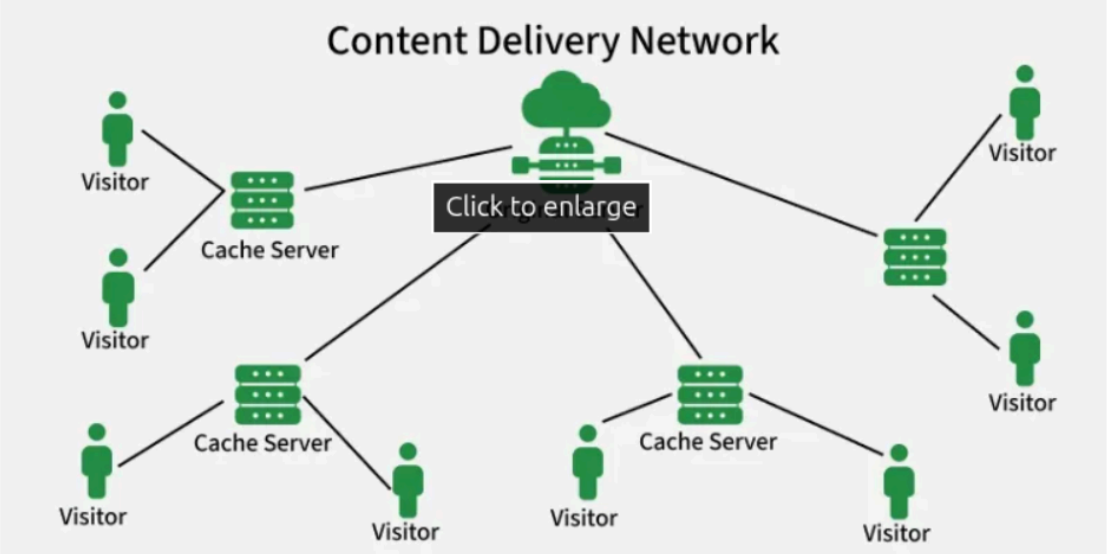
Speaker Notes: "Connect this to load balancer health checks, but note the checked unit here is an entire region, not one server. The TTL trade-off between fast failover and resolver load is a common interview probe; TTL also governs how fast cached answers expire."

Slide 20: CDN Edge Caching

Heading: CDN Edge Caching

Subheading: Content Near Users

Content: "A CDN keeps copies of content on servers called edge locations, placed close to users, so most requests are answered nearby instead of at a distant origin server."



- Edge location: a cache server near users; a cache hit means it already holds the content and replies at once.
- Cache miss: the edge lacks the content, fetches it from the origin and keeps a copy for the next user.
- Serving from the edge cuts delay for distant users and shields the origin from repeated requests.
- Static files such as images, style sheets and scripts cache well; personalised pages cache poorly.

Analogy: "A neighbourhood shop stocks popular items so customers need not travel to a distant factory every time — just as an edge location holds popular content and only the first request travels to the origin."

Visual Guidance: World map with three users linked to nearby edge locations and one long arrow to a single origin; green tick on a hit, amber arrow on a miss.

Speaker Notes: "Define cache hit and miss here, since the next slide builds on both. Name CloudFront as the AWS CDN. A common interview question is why a CDN helps even when the app runs in one region: distance and origin offload."

Slide 21: Refreshing Content From the Origin

Heading: Origin Pull Mechanism

Subheading: Refreshing Edge Content

Content:

An edge server must decide if its cached copy is still fresh, or stale enough to fetch again from the origin, the authoritative content server.

- A cache miss triggers an origin pull, fetching the object fresh.
- Time to Live (TTL), set via a cache-control header, decides how long an edge copy stays valid.
- A conditional request confirms a cached copy is still fresh without re-downloading it.
- Static assets get long TTLs; frequently changing pages get short or no caching.

Visual Guidance: Flow arrow: edge server cache box, a branch to origin server labelled origin pull on miss, and a loop back labelled TTL check on hit.

Speaker Notes: "Build on the known cache hit/miss idea, noting the miss path is called an origin pull. Explain TTL as the timer deciding when a cached copy turns stale. Exams often ask why static files get long TTLs and dynamic pages do not."

Slide 22: Forcing Edge Content to Refresh

Heading: Cache Invalidation

Subheading: Forcing Fresh Content

Content:

Waiting for a TTL to expire can leave stale content live for minutes or hours. Cache invalidation lets an operator remove or bypass it immediately.

- Purge invalidation removes an object from every edge before TTL expiry. CloudFront calls this an invalidation request.
- Versioned URLs attach a new filename or query string, skipping old cached files.
- A single invalidation request can target one path or an entire cache.
- Over-invalidating increases origin load since every edge must re-fetch content.

Visual Guidance: Two side-by-side mini-diagrams: a purge command striking a cached object, and a URL changing from style.css to style-v2.css beside an unchanged cached copy.

Speaker Notes: "Present purge and versioning as two answers to the same problem: stale content after an update. The versioned-URL trick is a favourite interview question since it needs no purge call. Note that aggressive purging pushes traffic back onto the origin."

Slide 23: DNS Routing Against Load Balancing

Heading: DNS Vs Load Balancer

Subheading: Two Routing Layers

Content:

DNS routing and a load balancer both redirect traffic from a failed target, but they work at different layers and speeds. Production systems usually combine both.

Layer	Scope	Failover Speed	Example
DNS routing	Across regions or data centres	Minutes, bound by record TTL	Geo or latency routing to the nearest region
Load balancer	Within one region, across servers	Seconds, by health-check interval	Application Load Balancer and its target group

DNS first sends a user to a healthy region, and a load balancer inside that region spreads requests across healthy servers. The two layers complement rather than replace each other.

Visual Guidance: Layered diagram: top layer labelled DNS routing across three region icons, bottom layer zoomed into one region showing a load balancer fanning out to three servers.

Speaker Notes: "Use the table to fix the layer and speed difference; this is a common interview question. DNS failover is slower due to TTL caching, while a load balancer reacts within its health-check interval. Both layers are usually combined, not chosen between."

---------- PRACTICAL HANDS-ON: Building a Load-Balanced Web Tier ----------

Slide 24: Building a Load-Balanced Web Tier

Heading: Load Balancer Build

Subheading: Task And Steps

Content:

This exercise builds a small load-balanced web tier: two web servers behind one Application Load Balancer, using the VPC and subnets created earlier. It creates billable AWS resources, so the Billing dashboard should be checked first, and all resources are deleted right after screenshots are captured.

An Application Load Balancer spans more than one Availability Zone, so the servers it balances across must sit in separate zones for true redundancy.

Resource	Expected Name	Expected Configuration
VPC	s5-cloud-network-vpc	CIDR 10.0.0.0/16
First public subnet	s5-public-subnet	10.0.1.0/24, Availability Zone ap-south-1a
Second public subnet	s6-public-subnet-2	10.0.3.0/24, Availability Zone ap-south-1b, associated with s5-public-rt
Internet Gateway	s5-igw	Attached to s5-cloud-network-vpc
Public route table	s5-public-rt	Default route 0.0.0.0/0 points to s5-igw
Private subnet	s5-private-subnet	10.0.2.0/24

1. Verify the Session 5 VPC, subnets, and route table against the table above before creating anything new.
2. Add one new subnet in a second zone, plus a load balancer security group and a web server security group.
3. Launch two EC2 web servers, one per public subnet, each running a startup script.
4. Create a target group and register both web servers as its targets.
5. Create an internet-facing Application Load Balancer spanning both subnets, forwarding to the target group.
6. Test the load balancer, then stop and restart a server to observe health checks.
7. Verify every component, capture screenshots, and delete resources in order: load balancer, target group, EC2 instances, s6-web-sg, s6-alb-sg, then s6-public-subnet-2.

Slide 25: Confirming Setup Before Testing

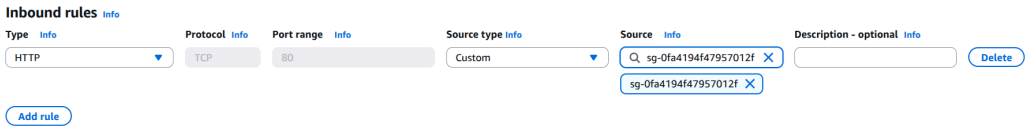
Heading: Setup Confirmation

Subheading: Security Groups, Listener, Servers

Content:

Before testing, confirm three things: the web-server security group only trusts the load balancer, the listener forwards to the right target group, and both servers are running.

Web Server Security Group



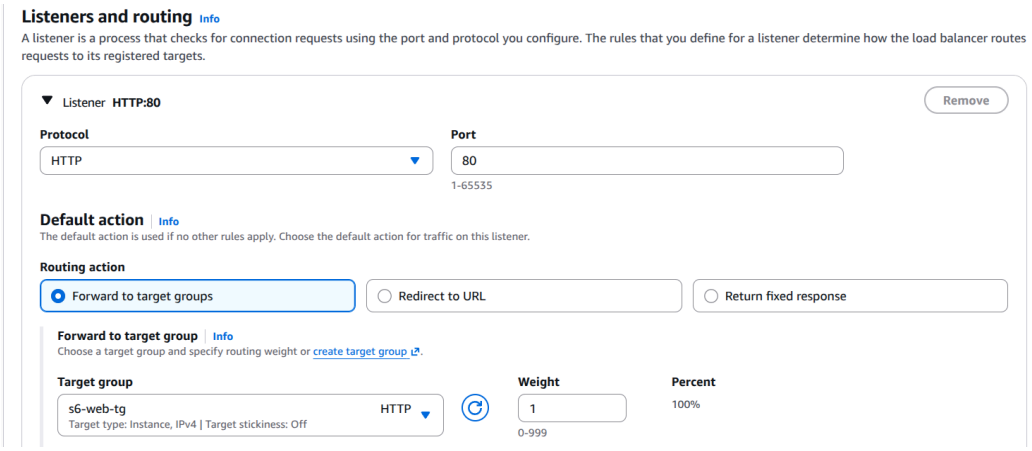
s6-web-sg inbound rule — source is the ALB's own security group, not the public internet

Load Balancer Security Group



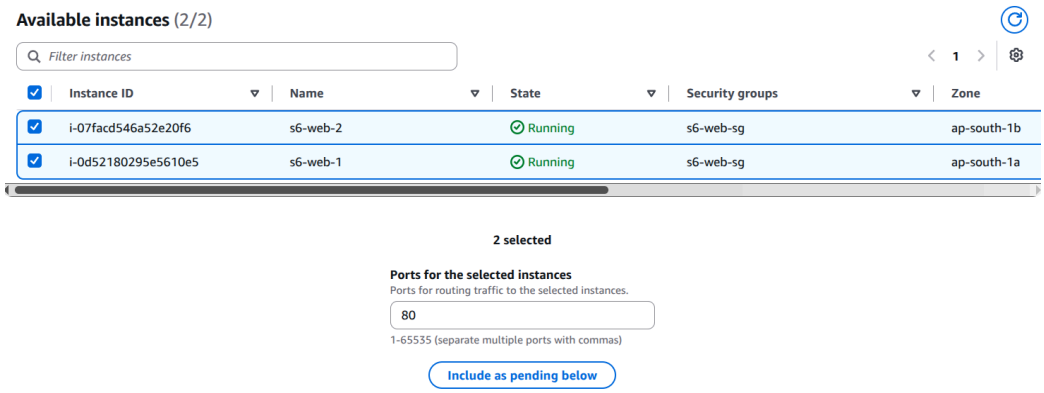
s6-alb-sg inbound rule — source is 0.0.0.0/0, open to all public traffic

Listener Routing



HTTP:80 listener forwards to target group s6-web-tg

Both Servers Running



Both EC2 instances (s6-web-1, s6-web-2) in Running state, ready to be registered

Slide 26: Testing Distribution And Health-Check Recovery

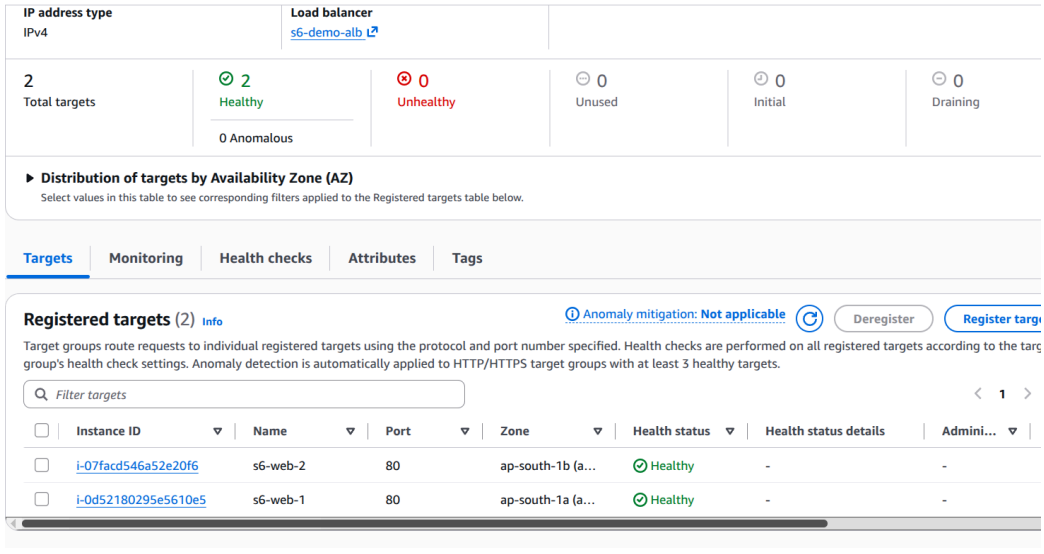
Heading: Load Balancer Action

Subheading: Test And Recovery

Content:

Testing confirms three behaviours: healthy targets share traffic, a stopped target is taken out of rotation, and a restarted target returns once it passes its health checks.

Both Targets Healthy

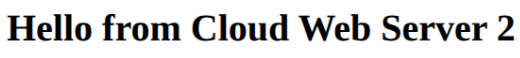


Target group summary — 2 Healthy, 0 Unhealthy, before any traffic test

Browser Test — Load Balancer Serves Both Servers

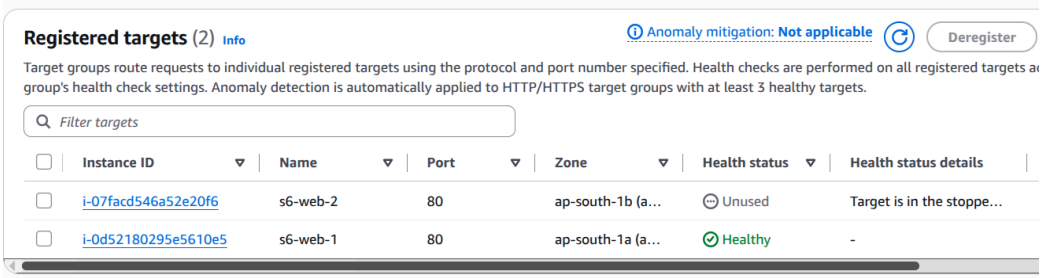


Request via the ALB's DNS name — answered by s6-web-1



A later request — answered by s6-web-2; which server answers is not guaranteed to alternate strictly

After Stopping s6-web-2



Status shows Unused — "Target is in the stopped state," not Unhealthy; Unhealthy is reserved for a running instance that fails its health check path

Slide 27: Expected Result and Wrap-Up

Heading: Result And Cleanup

Subheading: Wrap-Up Checklist

Content:

The finished setup is a single load balancer domain name that reaches two interchangeable web servers, each able to fail and recover without any manual rerouting of traffic.

Component	Expected Result
VPC	s5-cloud-network-vpc
Public subnets	s5-public-subnet and s6-public-subnet-2 in different zones
Security groups	s6-alb-sg open to 0.0.0.0/0; s6-web-sg open only to s6-alb-sg
EC2 instances	s6-web-1 and s6-web-2, both running
Target group	s6-web-tg, both instances Healthy
Load balancer	s6-demo-alb, listener forwards to s6-web-tg

This classroom setup enables public addresses on both backend servers to keep access simple; in a production deployment, servers sitting behind an internet-facing load balancer are usually placed in private subnets instead, since the load balancer is the only component that needs to be publicly reachable.

Cleanup runs in dependency order: the load balancer is deleted first, then the target group, then the two instances are terminated, then s6-web-sg, s6-alb-sg and s6-public-subnet-2, since each resource is still used by the one deleted before it.

Key Takeaways

1. Load Balancing Basics: Purpose: availability & scalability; the single-point-of-failure problem
2. Load Balancer Types: Layer 4 (transport) vs Layer 7 (application); internal vs internet-facing
3. Distribution Algorithms: Round robin; least connections; IP hash; weighted distribution
4. Health Checks & Sessions: Health check design; draining an unhealthy backend; sticky sessions vs stateless design
5. DNS & Content Delivery: DNS-based routing & failover; CDN: edge caching, origin & cache invalidation

Upcoming Session : Block, File & Object Storage

Thank You | All the Best